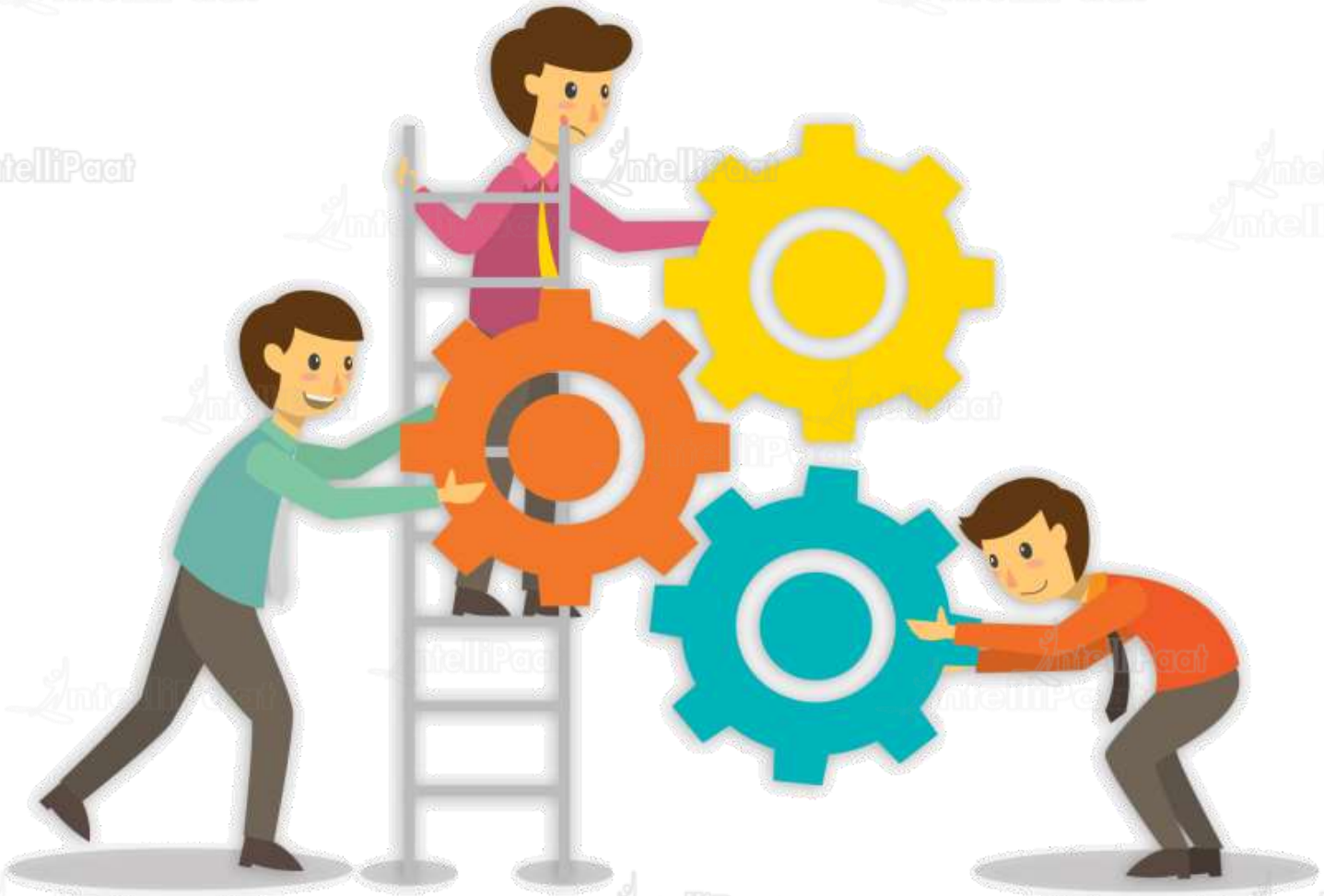
# Python Training

## Python Packages



Copyright Intellipaat. All rights reserved.

# Agenda

01 Python Packages

02 Packages Examples

03 Various ways to access Packages

04 ‘From…Import’ in Packages

05 From…Import* in Packages

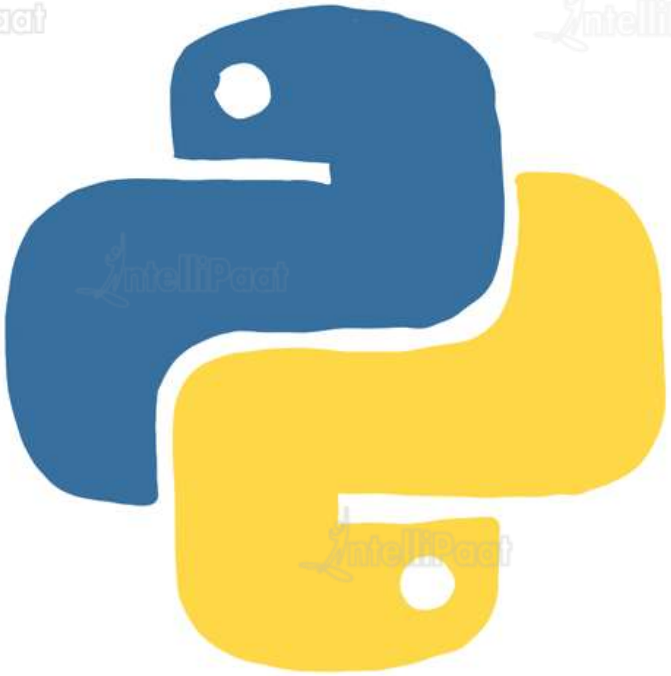
Copyright Intellipaat. All rights reserved.

# Python Packages

Copyright Intellipaat. All rights reserved.

# Python Packages

- Packages are a way of structuring many packages and modules which helps in a well-organized hierarchy of data set, making the directories and modules easy to access.
- Just like there are different drives and folders in an OS to help us store files, similarly packages help us in storing other sub-packages and modules, so that it can be used by the user when necessary.
- To tell Python that a particular directory is a package, we create a file named __init__.py inside it and then it is considered as a package and we may create other modules and sub-packages within it.

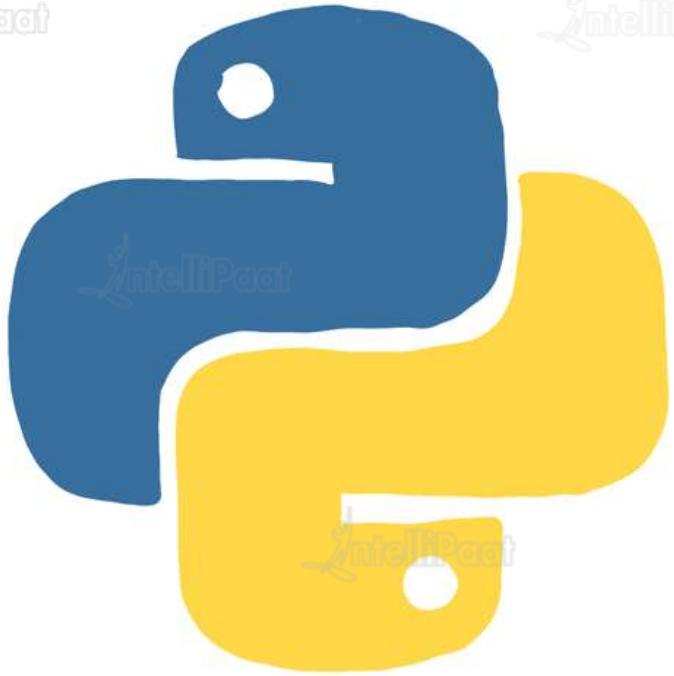


Packages

Copyright Intellipaat. All rights reserved.

# Python Packages

- This __init__.py file can be left blank or can be coded with the initialization code for the package.
- To create a package in Python, we need to follow these three simple steps:
- First, we create a directory and give it a package name, preferably related to its operation.
- Then we put the classes and the required functions in it.
- Finally we create an __init__.py file inside the directory, to let Python know that the directory is a package.

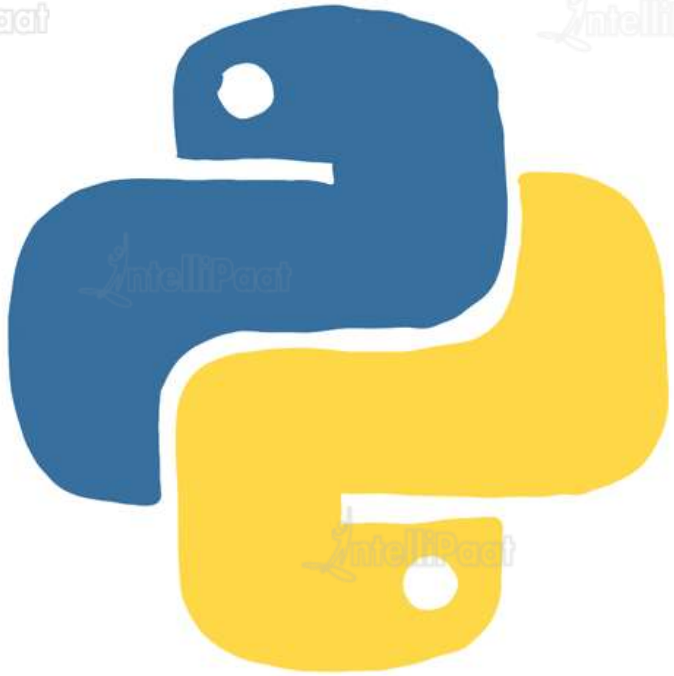


Packages

Copyright Intellipaat. All rights reserved.

# Python Packages

- Suppose you have developed a very large application that includes many modules.
- As the number of modules grows, it becomes difficult to keep track of them all if they are dumped into one location.
- This is particularly so if they have similar names or functionality.
- You might wish for a means of grouping and organizing them.
- Packages allow for a hierarchical structuring of the module namespace using dot notation.
- In the same way that modules help avoid collisions between global variable names, packages help avoid collisions between module names.

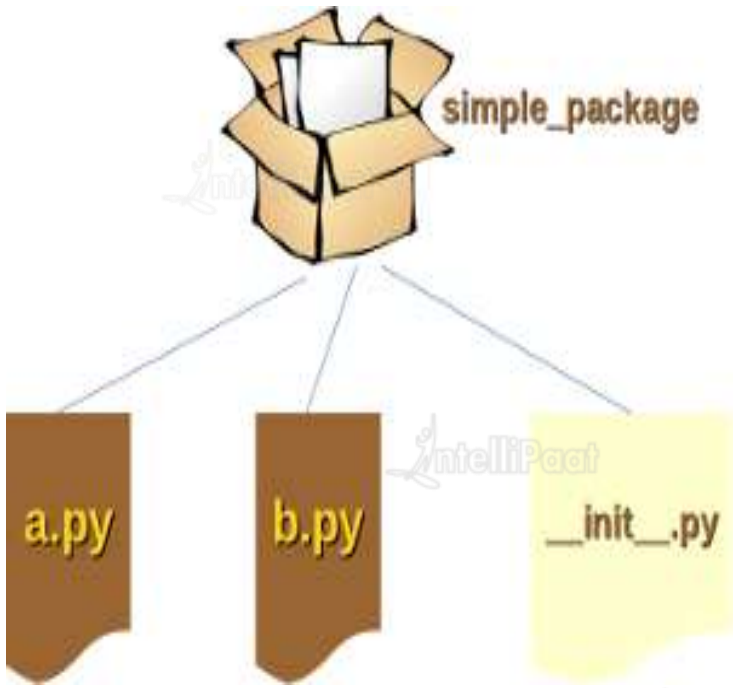


Packages

Copyright Intellipaat. All rights reserved.

# Python Packages

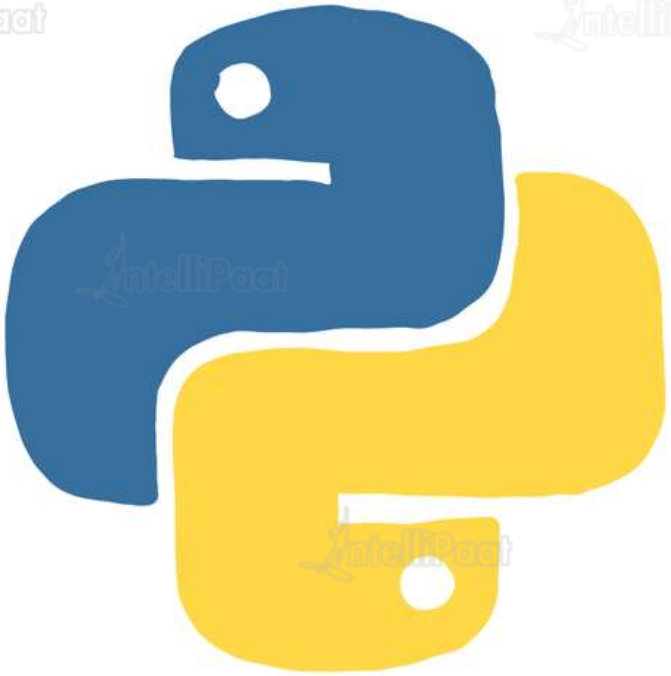
- We will demonstrate with a very simple example how to create a package with some Python modules.
- First of all, we need a directory. The name of this directory will be the name of the package, which we want to create.
- We will call our package "simple_package". This directory needs to contain a file with the name "__init__.py".
- This file can be empty, or it can contain valid Python code.
- This code will be executed when a package will be imported, so it can be used to initialize a package, e.g. to make sure that some other modules are imported or some values set.




Copyright Intellipaat. All rights reserved.

# Various ways of accessing the Packages

- Suppose the cars and the brand directories are packages.
- For them to be a package they all must contain __init__.py file in them, either blank or with some initialization code.
- Let’s assume that all the models of the cars to be modules.
- Use of packages helps importing any modules, individually or whole.
- Suppose we want to get Bmw i8. The syntax for that would be:
- 'import' Cars.Bmw.x5
- While importing a package or sub packages or modules, Python searches the whole tree of directories. looking for the particular package and proceeds systematically as programmed by the dot operator.



Packages

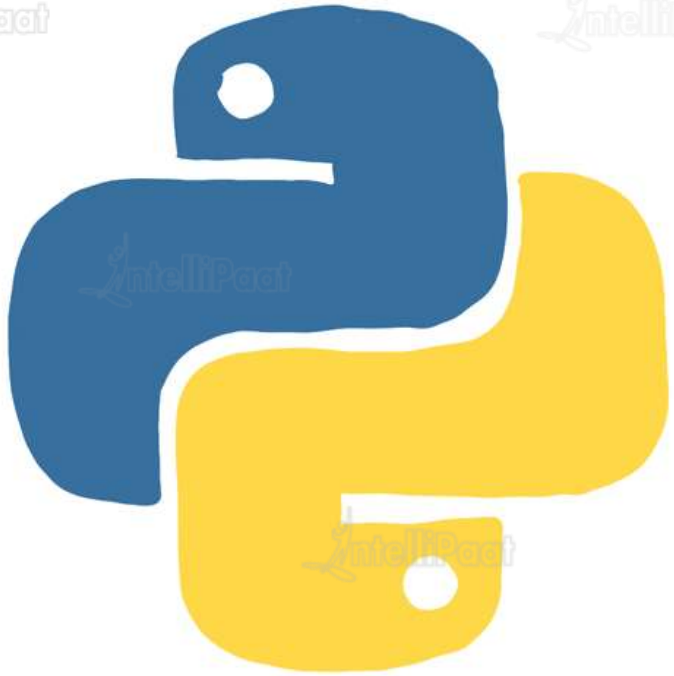
Copyright Intellipaat. All rights reserved.

# ‘From…Import’ in Packages

- Now, whenever we require using such function we would need to write the whole long line after importing the parent package.

- To get through this in a simpler way we use ‘from’ keyword.

- For this we first need to bring in the module using ‘from’ and ‘import’:
  from Cars.Audi import a8
- Now we can call the function anywhere using
  a8.get_buy(1)
- There’s also another way which is less lengthy. We can directly import the function and use it wherever necessary. First import it using:
  from Cars.Audi.a8 import get_buy
- Now call the function from anywhere:
  get_buy(1)

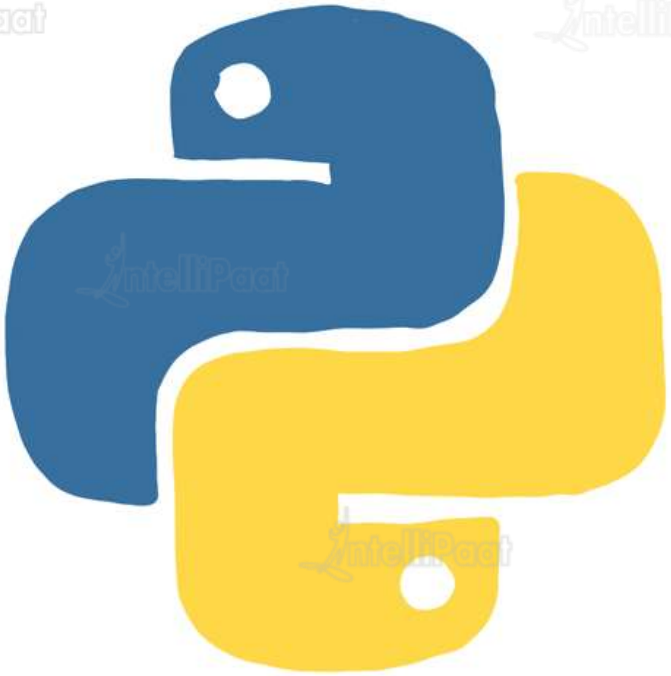


Packages

Copyright Intellipaat. All rights reserved.

# ‘From…import*’ in Packages

- While using the from…import syntax, we can import anything from submodules to class or function or variable, defined in the same module.
- If the mentioned attribute in the import part is not defined in the package then the compiler throws an ImportError exception.
- Importing sub-modules might cause unwanted side-effects that happens while importing submodules explicitly. Thus we can import various modules at a single time using * syntax.
- The syntax is:
  Impofrom Cars.Chevrolet import *
- This will import everything i.e., modules, sub-modules, function, classes, from the sub-package.
- While using just the import syntax, one must keep in mind that the last attribute must be a subpackage or a module, it should not be any function or class name.



Packages

Copyright Intellipaat. All rights reserved.

India: +91-7847955955

US: 1-800-216-8930 (TOLL FREE)

support@intellipaat.com

24/7 Chat with Our Course Advisor

Copyright Intellipaat. All rights reserved.